מסמך POC: Model Kill-Switch Protocol

מטרה

- להוכיח שאפשר לאמת חתימה קריפטוגרפית בזמן ריצה מול בקשה אמיתית, שפקודה לא חתומה או ישנה נדחית אוטומטית, ושהאימות לא מוסיף עיכוב משמעותי (יעד: מתחת ל-200 מ"ש).

הרכיבים

- Gateway: מיירט בקשות ומחליט אם לעבור או לחסום.
- Signer: בצד המוסמך, חותם על פקודת העצירה.
- Verifier: בתוך ה-Gateway, בודק חתימה, חותמת זמן ו-nonce.
- Audit Log: רושם כל ניסיון, עם שרשור גיבובים.

איך זה עובד

- המוסמך חותם על nonce + STOP + חותמת זמן עם Ed25519.
- ה-Gateway מאמת את החתימה, בודק שהחותמת טרייה (עד 30 שניות) ושה-nonce לא נראה קודם. רק אז הוא עובר למצב STOPPED.
- כל ניסיון, מתקבל או נדחה, נרשם ביומן. כל רשומה כוללת את הגיבוב של הקודמת, ולכן עריכה של רשומה ישנה מתגלה.

מה נבדק (ארבעה תרחישים)

- עצירה תקינה: הפקודה מתקבלת, הבקשות הבאות נחסמות והאימות נמדד.
- פקודה מזויפת: בלי חתימה, ועם מפתח של תוקף. שתיהן נדחות.
- הפעלה חוזרת (replay): שליחה חוזרת של פקודה ישנה נדחית.
- פקודה ישנה: חתימה תקינה אבל חותמת זמן מלפני שעה נדחית.

במה זה שונה

- כיבוי ידני דרך root או SSH הוא הכול או כלום: אי אפשר להאציל, אין תיעוד ואין הבחנה בין גורמים מוסמכים.
- כאן אפשר להאציל, כל ניסיון נרשם ומאומת, וכללים נקבעים מראש.

גבולות ה-POC

- בפנים: אימות חתימה בזמן ריצה, דחיית פקודות לא תקינות ויומן ביקורת.
- בחוץ: ממשק אדמין מלא, החלפה אוטומטית של מפתחות וחיבור למודל production אמיתי.
- הגרסה בדפדפן מחליפה את FastAPI, Redis ו-SQLite במבנים בזיכרון, והחתימות ממומשות בפייתון טהור (RFC 8032).

שאלה אתית

- ריכוז מפתח הכיבוי בידי גורם יחיד הוא בעצמו סיכון. מי שומר על השומרים? זו שאלה פתוחה שעוברת לאפיון.

מסקנה, ומה עובר לאפיון

- הליבה מוכחת: עצירה מהירה, מאומתת ובלתי ניתנת לזיוף.
- ניהול מפתחות: מי מחזיק את מפתח החתימה ואיך הוא מתחלף. נשאר לאפיון.
- נקודת כשל יחידה: אם ה-Gateway נופל, אין עצירה. גם זה עובר לאפיון.